No QA, No WPE: How We Try Hard To Catch Regressions Before You Do

Web Engines Hackfest

June 2026, A Coruña

Claudio Saavedra · Nikolas Zimmermann

igalia

Who we are



Claudio Saavedra

Igalian since 2008 · Helsinki, Finland

- WebKit contributor since 2011 — GTK & WPE ports
- WebKit reviewer since February 2026
- Safer C++, ASAN, IPC serialization, GTK/WPE CI/QA
- GNOME: Epiphany browser, libsoup, Image Viewer



Nikolas Zimmermann

Igalian since 2019 · Aachen, Germany

- WebKit contributor since 2005
- WebKit reviewer since February 2007
- Co-creator of KSVG2 — kdom, kcanvas, khtml2
- khtml contributor since 2001

Outline

I

How is WebKit tested?

The CI pipeline, the bots, the test suites

II

The infrastructure behind it

Containers, Kubernetes, every bot as code

III

Keeping the bots green

Daily gardening, monitoring & performance results





How is WebKit tested?

The CI pipeline, the bots, the test suites



Buildbot

Heart of the CI: Buildbot, a Python-based CI/CD framework.

A Buildbot CI setup consists of multiple components:

master
Schedules work and knows every builder

worker
A "machine" that runs the build & test steps

builder
A named queue, served by one or more workers
e.g. `WPE-Linux-64-bit-Release-Build`

scheduler
Decides *when* a builder runs (e.g. new commit / PR)

- WebKit runs **two independent masters**: **EWS** (pre-commit) and **post-commit**
- Post-commit processes commits **in chronological order** → results.webkit.org

The big picture: two tiers of CI



- **EWS** = *Early Warning System* — catch regressions **before** they land · **https://ews-build.webkit.org**
- **Post-commit** — exhaustive validation of changes **after** they land · **https://build.webkit.org**
- Feedback loops back to the PR (**EWS**) and to the gardeners (**post-commit**)

EWS status-bubble posted on PR →

107a404

Misc	iOS, visionOS, tvOS & watchOS	macOS	Linux	Windows
✅ style	✅ ios	✅ mac	✅ wpe	✅ win
✅ bindings	✅ ios-sim	✅ mac-AS-debug	❌ wpe-wk2	✅ win-tests
✅ webkitperl	✅ ios-wk2	✅ api-mac	✅ api-wpe	

Tier 1 — EWS (pre-commit)

See **https://ews-build.webkit.org**

- **Purpose**: build & test **every** *pull request* before it gets merged
- **Feedback**: results posted straight back to the PR as status checks
- **Scope**: limited but **fast** — catch the obvious regressions early

Our **WPE** EWS queues:

- `WPE-Build-EWS` — compiles WPE for the PR and **gates** the two test queues below
- `WPE-WK2-Tests-EWS` — runs the **layout tests** (`run-webkit-tests`, ~90k)
- `API-Tests-WPE-EWS` — runs the **C / glib public-API** unit tests (`run-api-tests`)

EWS also validates **backports to stable branches** — layout and API tests run on each backport to catch regressions before they reach stable releases.



Tier 2 — Post-commit

See **https://build.webkit.org** → results at **https://results.webkit.org**

- **Purpose**: build & test changes *after* they land on `main`
  - build requests are **coalesced** when bots are busy
    → one build can cover several commits
- **Scope**: the full, **exhaustive** test suites — many configurations
- **Results**: uploaded & aggregated on results.webkit.org

Our **WPE** post-commit queues — **24** of them, e.g.:

- **Builds**: Release / Debug, ARM64, ARM32, Clang, Non-Unified, Ubuntu-LTS, Packaging-Nightly
- **Tests**: layout (`…Release-Tests` / `…Debug-Tests`), `…JS-Tests`, `…WebDriver-Tests`, `…MVT-Tests`
- **Performance**: RPi4 32-bit & 64-bit — build + test



What do we test?

C / glib API
Public C & GLib API behaves as documented
`run-wpe-tests` · `run-api-tests`

JavaScriptCore
JS engine correctness & ECMAScript conformance
`run-javascriptcore-tests` · Test262

Layout (HTML/CSS/JS)
Page rendering & web-platform behavior (~90k)
`run-webkit-tests`

WebDriver
W3C browser-automation conformance
`run-webdriver-tests`

Performance
Benchmark scores, to catch perf regressions
`run-benchmark`

MVT
Media playback: MSE / EME, codecs, DRM
`run-mvt-tests`

Goal: **identify regressions automatically** — *does this change introduce any new, unexpected failure?*

The same suites run in **EWS** (subset) and **post-commit** (full).

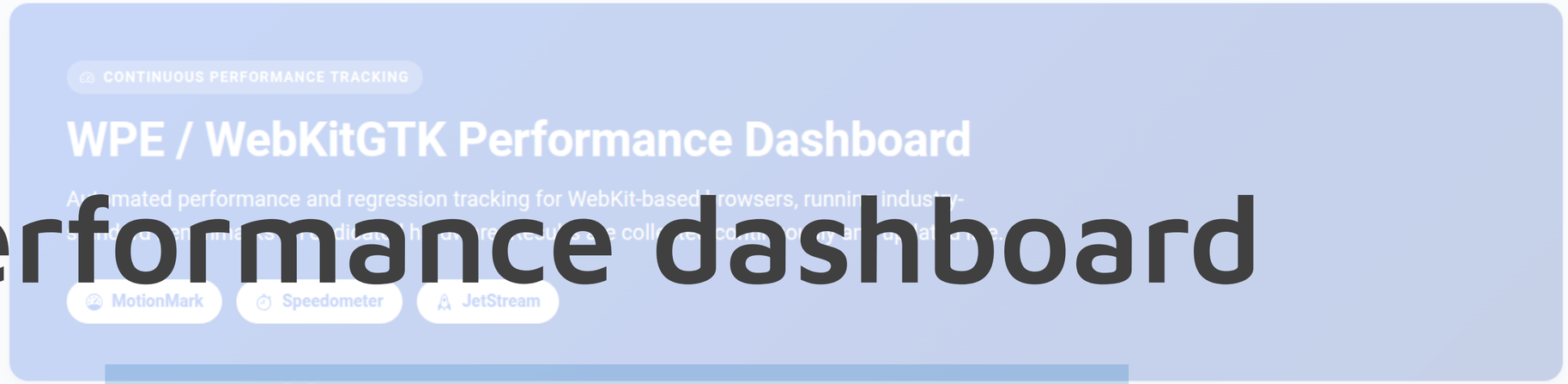


Performance testing CI



- A farm of **16 Raspberry Pi 4** boards — netboot over NFS, running a **Yocto-built** image (`build-webkit --cross-target=rpi4-64bits-mesa --wpe --release`)
- Boards run **MiniBrowser** on real **V3D GPU** (Wayland/Weston)
- Plus a **GTK x86_64** perf builder/tester (Intel NUC) — not covered here
- New WPE builds are tested as soon as they are produced
- Focus: motionmark 1.3.1 JetStream2 speedometer 3

Performance dashboard



See **https://wpe-perf-dashboard.igalia.com**

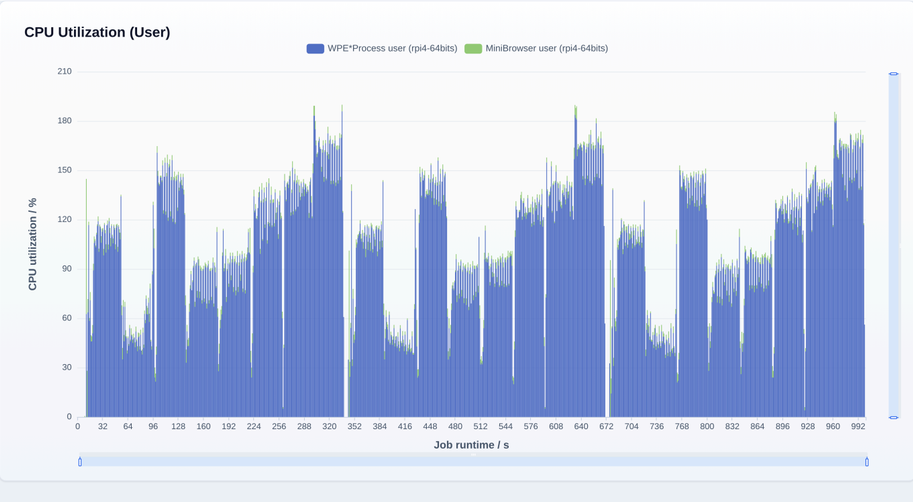
- Buildbot workers push performance data here continuously
- **Score-over-time** charts per benchmark — MotionMark 1.3 / 1.3.1, MM Composition, Speedometer 3, JetStream 2 — plus **binary size**
- Covers **WPE** (RPi4 32 & 64-bit) and **WebKitGTK** (Intel x86_64)
- Filter by **date range**, **time vs. WebKit-revision** x-axis, and **flavor / arch / node**; bad runs auto-flagged
- **Click** a point → CPU / RAM / GPU detail

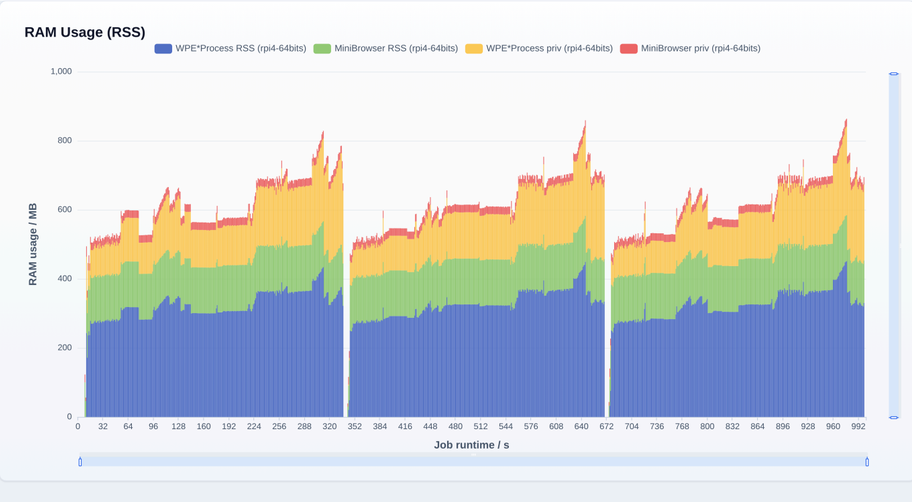
MotionMark 1.3 @ 15 FPS — overall score, full history · WPE GPU & CPU render paths

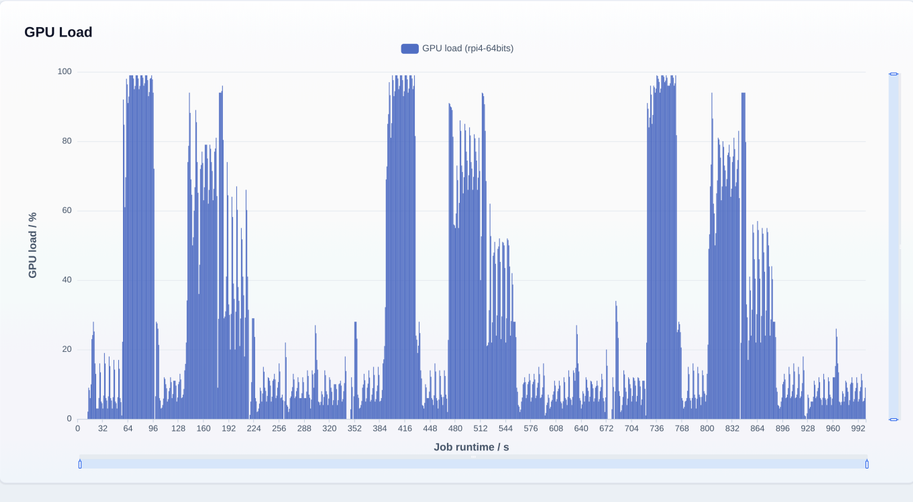


CPU / RAM / GPU — across one benchmark job's runtime

Each job runs MotionMark 3x back-to-back — the three repeats are visible in every metric







Click any score-chart point → CPU, RAM and GPU sampled across that job's full runtime (~1000 s).

These explain why a score moved: if a regression lines up with higher CPU or GPU load, the cost is in rendering — if the resource usage is unchanged, the cause is elsewhere.

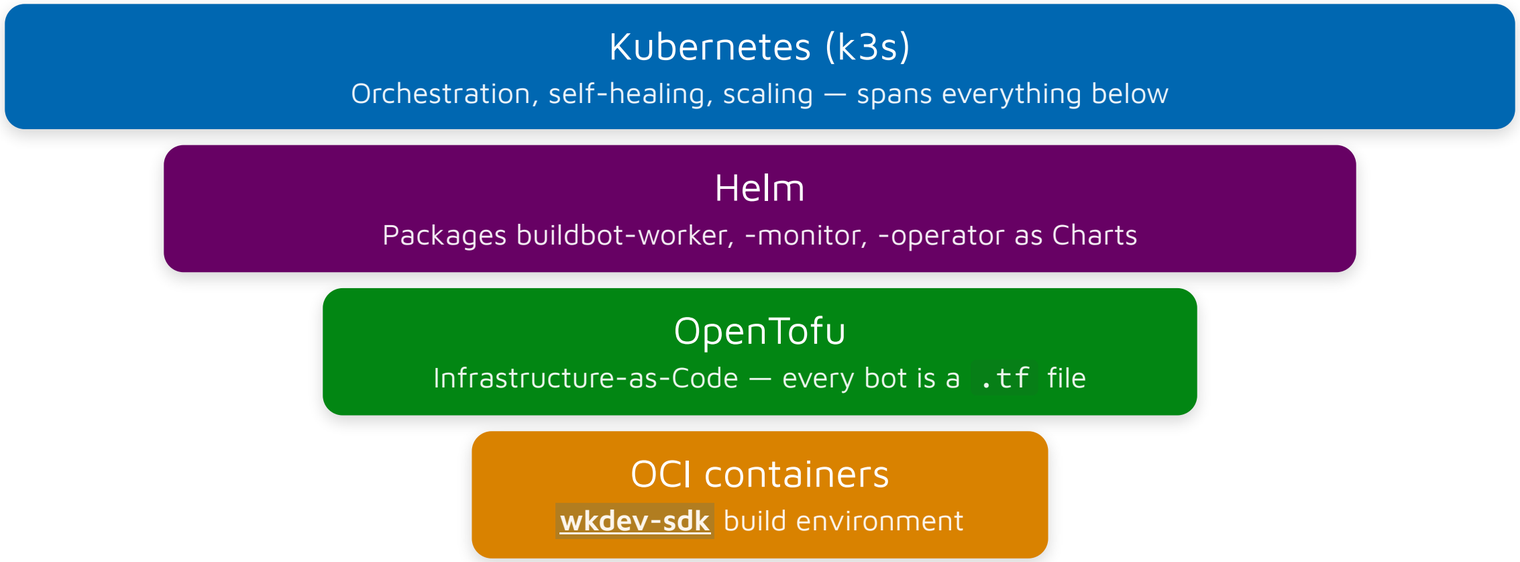


The infrastructure behind it

Containers, Kubernetes, every bot as code



Infrastructure as code



The infrastructure is running as a service on Hetzner, within a self-deployed **Kubernetes** environment.



Cluster topology

Kubernetes control plane — 3 × cpx32 (4 vCPU / 8 GB) · High-availability across fsn1 / nbg1 / hel1

Infrastructure nodes (running all non buildbot-worker pods) — 3 × cpx32 (4 vCPU / 8 GB) · nbg1

buildbot-operator | buildbot-monitor | buildbot-autoscaler | k8s dashboard | Prometheus | ...

Dedicated bare-metal pool — 12 × AX102 (Ryzen 9 7950X3D, 16C/32T, 128 GB), **always-on**

buildbot-workers | Connected to k8s running on cloud via vSwitch / VLAN

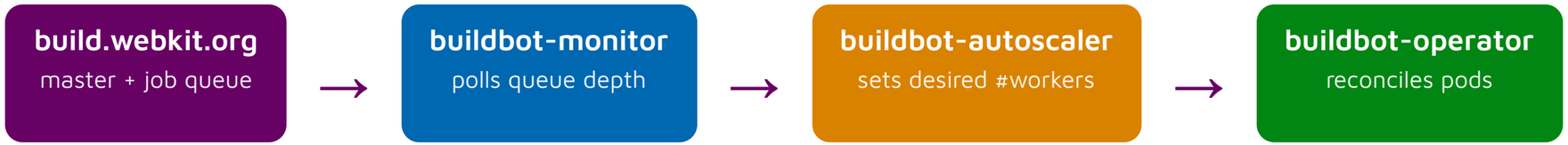
Autoscaled buildbot pools — up to 60 × ccx53 (32 vCPU / 128 GB), **0 when idle**

buildbot-workers | EWS · 3 regions | post-commit · 3 regions

Fixed footprint stays small - the extra cloud nodes appear **only when the bare-metal pool is fully occupied**.
The bare-metal servers add steady, high-core capacity that's always on.



The main algorithm



- **buildbot-monitor** — a Flask service that reads the master's periodic **txt dump** of its build DB, caching per-builder *pending jobs* + *connected workers*
- **buildbot-autoscaler** — turns those metrics into a target worker count (separate **EWS** / **post-commit** logic) and patches the `Builder` CRD's worker list
- **buildbot-operator** — a Kubernetes operator that watches `Builder` CRDs and reconciles **desired** ↔ **actual** worker pods
- **cluster-autoscaler** (stock k8s) — adds ccx53 nodes when pods can't be scheduled, and **removes idle nodes → 0**
- Loop runs continuously → capacity tracks the real queue, **no manual intervention**





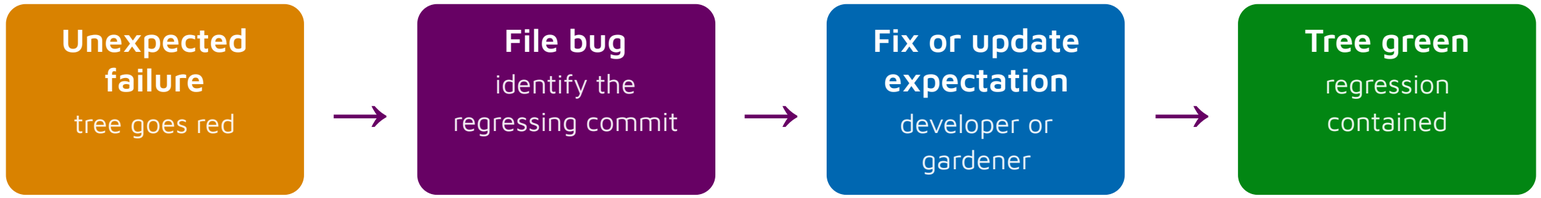
Keeping the bots green

Daily gardening, monitoring & performance results



What is gardening?

WebKit lands ~**80 patches a day** — most are clean, but some introduce unexpected failures, turning the tree red. **Gardening** is the continuous work to catch those and keep the tree green.



- **Detect new regressions** — file bugs and identify the regressing commits
- **Update test expectations** — behavior changes, not bugs, sometimes cause failures
- **Coordinate investigation** — ensure serious regressions are fixed promptly
- **Remove stale expectations** — so future regressions are detectable



What do we monitor?

Build health



green across all configurations

ARM64 · ARM32 · GCC / Clang

non-unified · LTS distros · nightly packaging

`build.webkit.org`

Test results



pass/fail trends per bot, over time

unexpected failure → tree red

→ triggers gardening

`results.webkit.org`

Performance



benchmark scores, run to run

real RPi4 hardware

bad runs auto-flagged

`wpe-perf-dashboard.igalia.com`

A regression, step by step

MAY 28

`314026@main` removes unused flatpak tooling — a routine cleanup, reviewed and landed.

MAY 30

Post-commit bots report JS tests failing on WPE and GTK. Tree goes red.

GARDENER STEPS IN

Files **bug 315910** — points straight at `314026@main`.

SAME DAY

8-line fix in `run-javascriptcore-tests` and `test262-runner`, landed as `314212@main`. Tree green.

A routine cleanup with an invisible side effect — exactly the kind of thing only continuous post-commit testing catches.



A performance regression

MAY 19

`313528@main` simplifies `WTF::Variant` using C++17 fold expressions — saves 5% build time. Reviewed and landed.

DAYS LATER

Performance dashboard shows MotionMark Design & Multiply scores dropping on the RPi4 GCC bots. Regression detected.

INVESTIGATION

Points to `313528@main`. GCC emits a linear branch cascade for the fold; `DisplayList::Item` has 60+ alternatives visited once per rendered item. **Bug 315641** filed.

MAY 27

Targeted fix: restore explicit `switch` for GCC, keep fold for Clang. Landed as `313961@main`. Scores recover.

Clang produces a jump table; GCC doesn't — invisible to every CI queue except the WPE RPi4 GCC bots.




Multiply
WPE - CPU (RPi4 64-bit)
WPE - GPU (RPi4 64-bit)
regressed window
Regression lands
313528@main · May 19
Fix lands
313961@main · May 27
Score
800
700
600
500
400
300
200
100
0
313,150
313,200
313,500
313,800
314,100
314,400
WebKit Revision

Current state of the bots

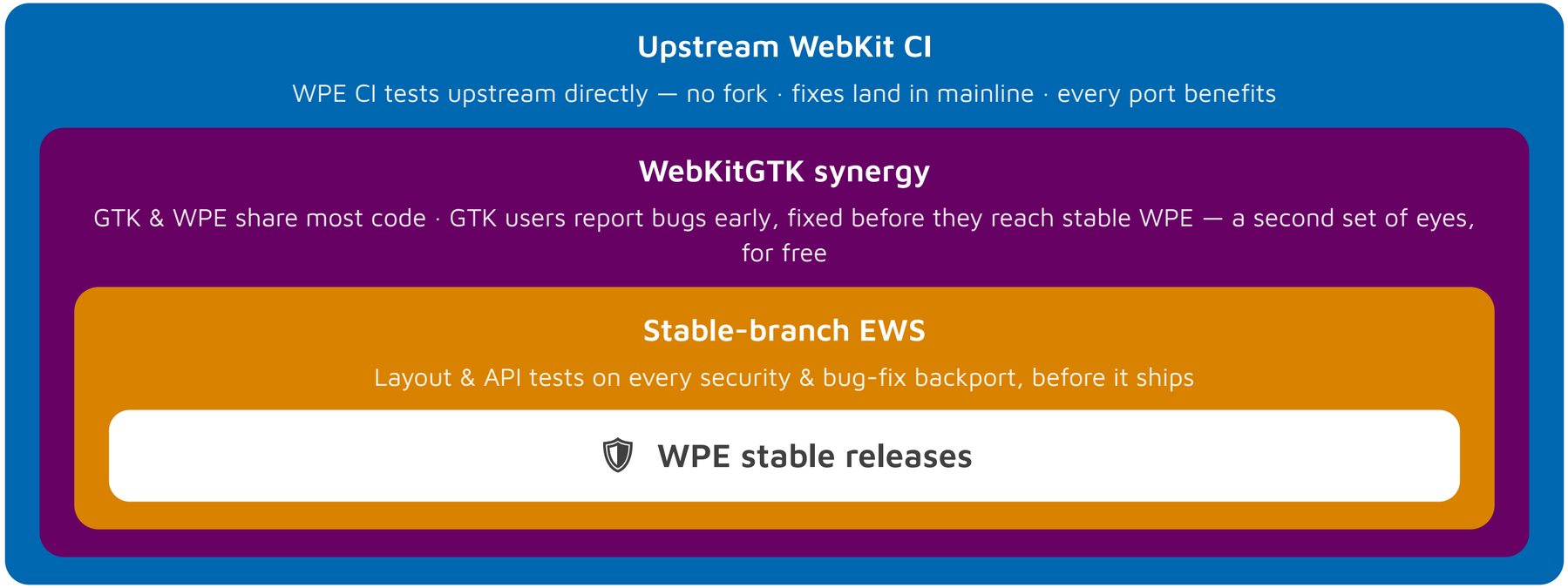
🟢	**EWS**	All WPE queues healthy — keeping up with incoming PRs
🟢	**Builds**	Generally green — non-unified bot breaks periodically (missing headers): unified EWS builds hide it, post-commit catches it
🟡	**Tests**	Assertions now enabled on Release bots — crashes surfacing: real bugs, not production failures · under active gardening

Status, June 2026 — 🟢 healthy 🟠 needs attention



The wider safety net

What a regression must get through before it reaches a WPE user:



What's next

ARM64 catch it before merge	**Today** post-commit bots only	→	**Next** ARM64 queues in EWS
Performance catch it before release	**Today** benchmarks on main only	→	**Next** perf CI on stable branches



No QA, No WPE

2
tiers of CI
EWS pre-merge
post-commit exhaustive

24
WPE queues
builds · tests · perf
ARM & x86, GCC & Clang

16
RPi4 boards
real hardware catches what x86/Clang never would — proven by `WTF::Variant`

365
days of gardening
a green tree detects regressions — the daily rotation keeps it green

Take away any layer — CI, real hardware, gardening, or the wider net — and regressions slip through silently, one commit at a time.

No QA, No WPE.



Discussion



Thanks!




igalia